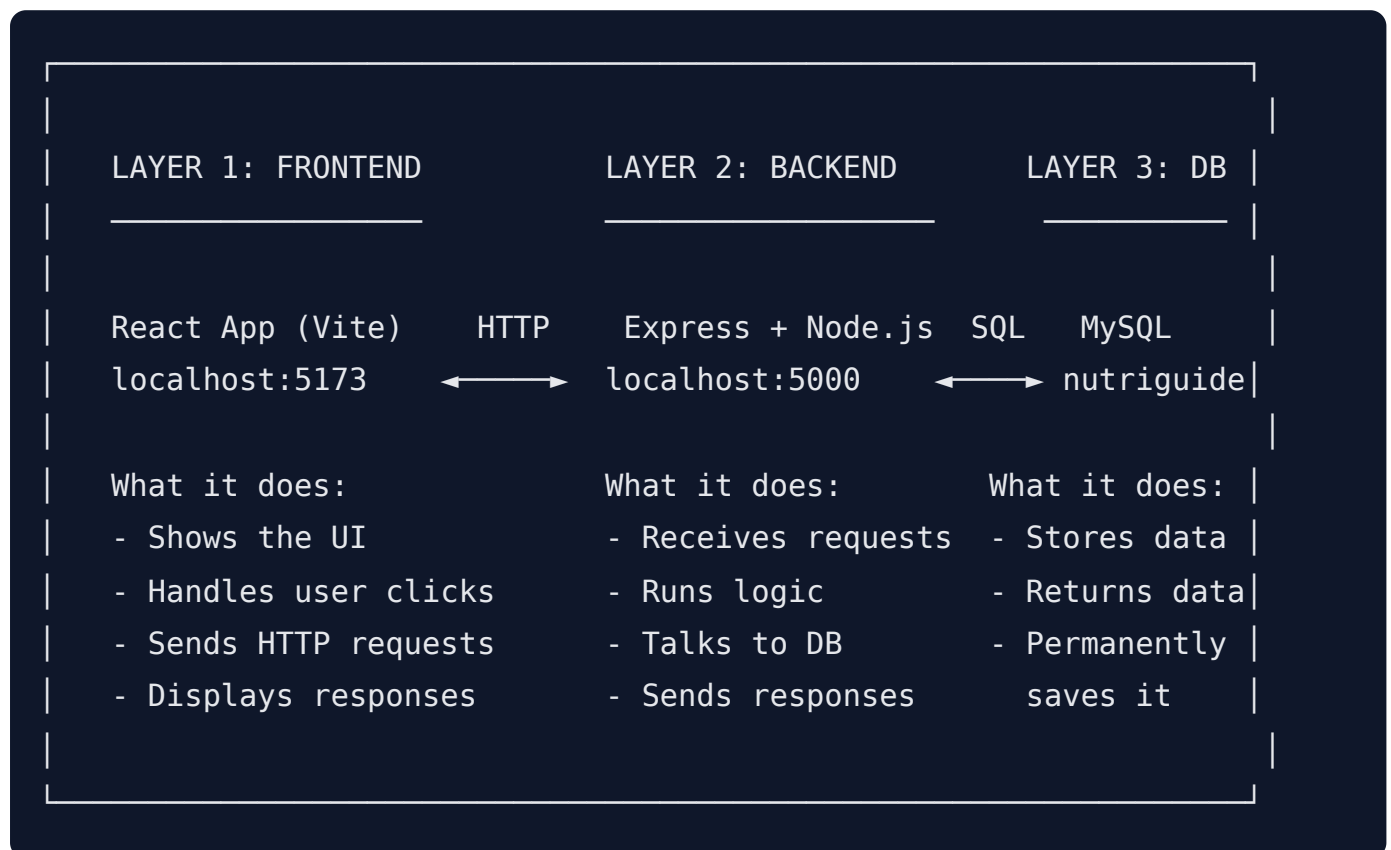


How Frontend, Backend & Database Are Connected

The Big Picture — 3 Layers



How Are They Physically Connected?

Frontend ↔ Backend: via HTTP (Axios)

The frontend and backend talk using **HTTP requests** — the same protocol your browser uses to load websites.

The frontend uses **Axios** (a JavaScript library) to send these requests.

```
// frontend/src/services/api.js

const api = axios.create({
  baseURL: 'http://localhost:5000/api' // ← THIS is how frontend
  knows where backend is
});
```

Every time a page needs data, it calls `api.get(...)` or `api.post(...)` and Axios sends an **HTTP request** to the backend at port `5000`.

Backend ↔ Database: via MySQL Connection Pool

The backend talks to MySQL using the **mysql2** library.

```
// backend/config/db.js

const pool = mysql.createPool({
  host: 'localhost',          // ← MySQL is on the same machine
  user: 'cdac',               // ← MySQL username
  password: 'cdac',           // ← MySQL password
  database: 'nutriguide',     // ← which database to use
  connectionLimit: 10        // ← up to 10 queries at once
});
```

Every controller imports this pool and runs SQL queries through it:

```
const [users] = await db.query('SELECT * FROM users WHERE email = ?',
  [email]);
```



The JWT Token — The Connection Glue

The **JWT token** is what ties all 3 layers together securely. Here's how:

1. User logs in → Backend creates JWT token (contains userId + role)
`jwt.sign({ id: 1, role: 'user' }, SECRET, { expiresIn: '24h' })`
2. Token sent to Frontend → stored in localStorage
`localStorage.setItem('token', 'eyJhbGc...')`
3. Every future request → Axios interceptor adds token automatically
Authorization: "Bearer eyJhbGc..."
4. Backend middleware reads token → extracts userId
`req.userId = 1` ← now backend knows WHO is making the request
5. Backend uses userId to query database
`SELECT * FROM users WHERE id = 1`



The Full Connection — Step by Step



JOURNEY 1: User Registers

FRONTEND (Register.jsx)

```
|
| User fills form: name, email, password, age, weight...
| Clicks "Register" button
|
| api.post('/auth/register', { name, email, password, age, weight, ...
| })
|
|           |
|           HTTP POST Request
|           http://localhost:5000/api/auth/register
|
```



BACKEND (server.js → authRoutes.js → authController.js)

```
|
| 1. server.js receives POST /api/auth/register
| 2. Passes to authRoutes.js
| 3. authRoutes calls authController.register()
|
| Controller logic:
| - Check if email already exists:
|   db.query('SELECT * FROM users WHERE email = ?', [email])
| - Hash the password:
|   bcrypt.hash(password, 10) → "$2b$10$Kh..."
| - Insert new user:
|   db.query('INSERT INTO users (name, email, password...) VALUES
| (...)' )
|
```



DATABASE (MySQL - users table)

```
|
| New row inserted:
| id=1, name="Moraya", email="moraya@gmail.com",
| password="$2b$10$Kh..." ← hashed!
| role="user", health_goal="lose_weight"
|
```



BACKEND → FRONTEND

```
|  
| Returns: { message: "User registered successfully", userId: 1 }  
|
```



FRONTEND

Receives success → shows "Registration successful!" → navigates to /login

JOURNEY 2: User Logs In

FRONTEND (Login.jsx)

```
|  
| User enters: email + password  
| Clicks "Login"  
|  
| api.post('/auth/login', { email, password })  
|           |  
|           HTTP POST Request  
|
```



BACKEND (authController.login)

```
|  
| 1. Find user by email:  
|   db.query('SELECT * FROM users WHERE email = ?',  
| ['moraya@gmail.com'])  
|   → returns: { id:1, name:"Moraya", password:"$2b$10$Kh...",  
| role:"user" }  
|  
| 2. Compare passwords:  
|   bcrypt.compare("userTyped1234", "$2b$10$Kh...") → true   
|  
| 3. Create JWT token:  
|   jwt.sign({ id:1, role:'user' }, SECRET, { expiresIn:'24h' })  
|   → "eyJhbGciOiJIUzI1NiJ9..."  
|
```



DATABASE

No write here – just a READ (SELECT)



BACKEND → FRONTEND

```
|  
| Returns: {  
|   token: "eyJhbGciOiJIUzI1NiJ9...",  
|   user: { id:1, name:"Moraya", email:"...", role:"user", weight:70...  
| }  
| }
```

```
|  
▼  
FRONTEND (AuthContext.jsx)  
|  
| login(userData, token) is called:  
| - localStorage.setItem('token', 'eyJhbGc...') ← saves token  
| - setUser({ id:1, name:"Moraya", ... }) ← sets global state  
|  
| App.jsx detects user is logged in  
| → Navigate to /dashboard ✓
```

✓ JOURNEY 3: Get Personalized Meal Recommendations

FRONTEND (Meals.jsx)

Page loads → needs meal recommendations for THIS user

```
api.get('/recipes/recommendations')
```

↑ Axios interceptor runs first:

reads localStorage → gets token

adds header: Authorization: "Bearer eyJhbGc..."



BACKEND (recipeRoutes.js → verifyToken → recipeController)

1. verifyToken middleware:

- Reads "Bearer eyJhbGc..." from header

- jwt.verify(token, SECRET) → decodes { id:1, role:'user' }

- Sets req.userId = 1

- Calls next() → passes to controller

2. recipeController.getRecommendations():



DATABASE (Query 1 – get user preferences)

```
db.query('SELECT diet_preference, allergies, health_goal,  
          weight, target_weight FROM users WHERE id = ?', [1])
```

```
→ { diet_preference:'vegetarian', health_goal:'lose_weight',  
    weight:70, target_weight:60, allergies:'nuts' }
```



DATABASE (Query 2 – get matching recipes)

```
db.query('SELECT * FROM recipes WHERE diet_tag = "vegetarian"')
```

```
→ [{ id:5, title:"Dal Tadka", calories:320, ... },  
    { id:8, title:"Paneer Curry", calories:450, ... }, ...]
```



BACKEND (in-memory filtering)

- Filter out recipes with "nuts" in ingredients (allergy check)
- health_goal = 'lose_weight' → filter calories ≤ 500
- Sort by calories ascending (lowest first)
- Return top 10

▼
BACKEND → FRONTEND

Returns: [{ id:5, title:"Dal Tadka", calories:320 },
 { id:3, title:"Moong Dal", calories:280 }, ...]

▼
FRONTEND (Meals.jsx)

Receives recipes → renders meal cards on screen 

JOURNEY 4: User Logs a Meal

FRONTEND (Meals.jsx)

```
|
| User clicks "Log Meal" on "Dal Tadka" card
|
| api.post('/meals/', {
|   recipe_id: 5,
|   date: '2026-05-09',
|   meal_type: 'lunch'
| })
| ↑ Axios automatically adds: Authorization: "Bearer eyJhbGc..."
|
```



BACKEND (mealRoutes.js → verifyToken → mealController.logMeal)

```
|
| 1. verifyToken → req.userId = 1
|
| 2. mealController.logMeal():
|   Reads from request:
|   - req.body.recipe_id = 5
|   - req.body.date = '2026-05-09'
|   - req.body.meal_type = 'lunch'
|   - req.userId = 1 ← from token!
|
```



DATABASE (INSERT into meal_logs)

```
|
| db.query(
|   'INSERT INTO meal_logs (user_id, recipe_id, date, meal_type)
|   VALUES (?, ?, ?, ?)',
|   [1, 5, '2026-05-09', 'lunch']
| )
|
| New row saved:
| id=23, user_id=1, recipe_id=5,
| date='2026-05-09', meal_type='lunch'
|
```



BACKEND → FRONTEND

|

| Returns: { message: "Meal logged successfully", logId: 23 }

|

▼

FRONTEND

Shows success toast / updates daily meal list 



Ports — Why 2 Servers Run Simultaneously

When you develop NutriGuide, you run **two servers at the same time**:

Terminal 1 (frontend): `npm run dev` → `http://localhost:5173`

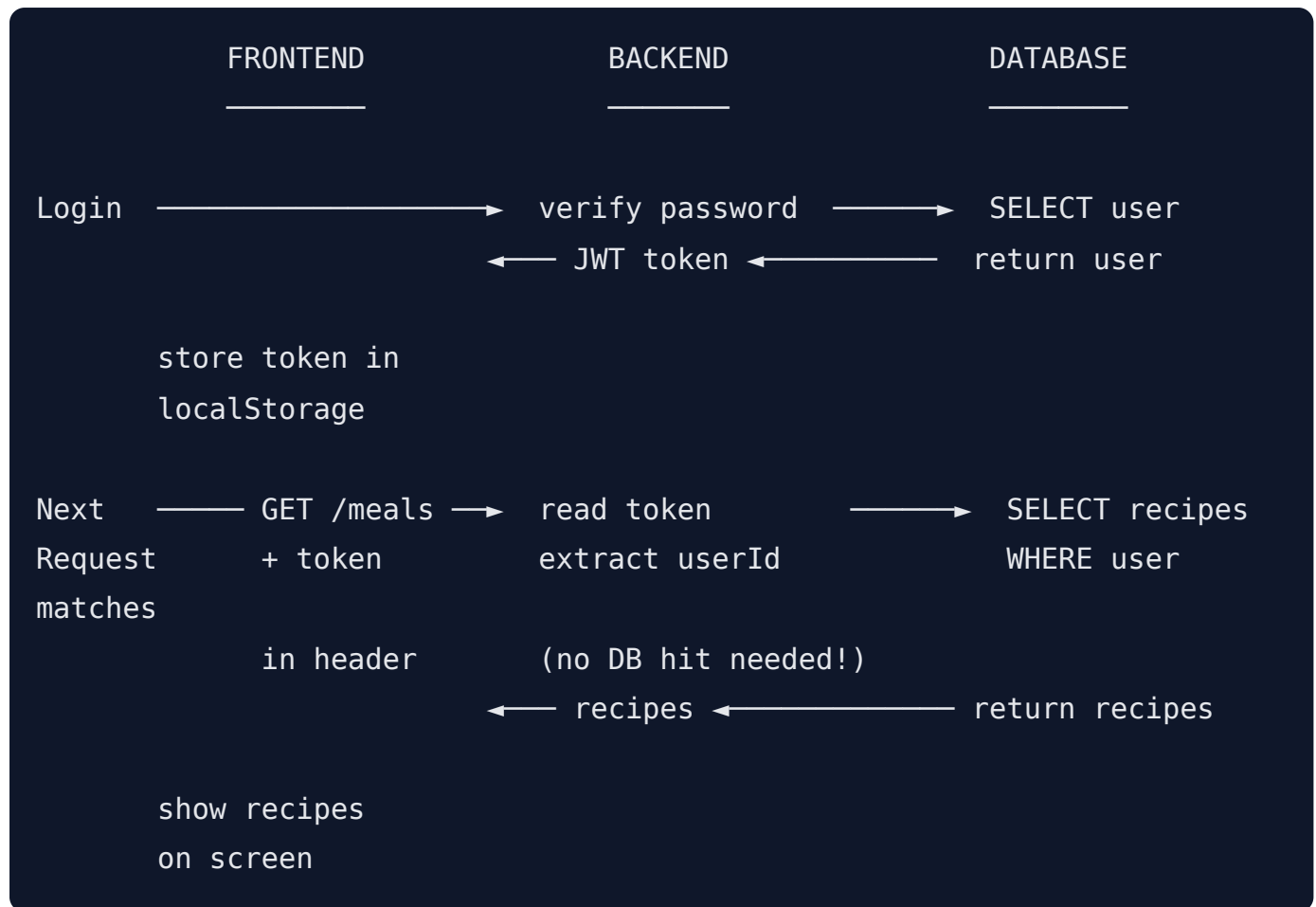
Terminal 2 (backend): `node server.js` → `http://localhost:5000`

Who	Where	What runs
Frontend	<code>localhost:5173</code>	Vite dev server serving React app
Backend	<code>localhost:5000</code>	Express server handling API requests
Database	<code>localhost:3306</code>	MySQL (default port, always running)

The **CORS** setting in `server.js` allows the frontend on `5173` to talk to the backend on `5000`.



The Token Flow — How All 3 Layers Stay in Sync



💡 **Key insight:** After login, the **userId is embedded inside the JWT token**. The backend never has to "look up who is logged in" — it just decodes the token. The database is only hit when actual data (recipes, meals) is needed.

Complete Connection Map

